Signals & Systems Lab

Coding Question Bank

Template-completion tasks in the style of Online A / B / C

Each question gives you a code template with a gap (a `raise NotImplementedError` stub, a `# TODO`, or a deliberate bug). You implement or fix the marked part so the program produces the required behaviour. A complete worked solution with reasoning follows every question.

Topics: NumPy signal construction • time scaling & interpolation • time shift & phase change • time reversal & even/odd decomposition • step/impulse synthesis • convolution • energy/power • plotting with Matplotlib

Difficulty tags: [basic] • [core] • [tricky]

How to use this bank

These questions follow the same contract as your online lab: a runnable file is provided, and you only change the marked region. Assume every file begins with the two standard imports and, where relevant, the given constants.

```
import numpy as np
import matplotlib.pyplot as plt

T_MIN, T_MAX, N = -4.0, 4.0, 4001     # (used by the continuous-time tasks)
```

For each task: (1) read the scenario, (2) study the template, (3) write only the missing/fixed lines, (4) check against the solution. Marks in the real exam usually come from correct implementation plus a properly labelled plot (title, x-label, y-label, legend, grid), so the solutions include those where a plot is asked for.

> Reminder. Nothing displays until `plt.show()` runs. Use `axis=1` for per-row and `axis=0` for per-column aggregation. Use `&` / `|` (not `and` / `or`) on NumPy boolean masks. Copy an array with `.copy()` before editing it in place.

Part 1 - Building Signals with NumPy

Warm-ups: create the time axis and standard signals used by every later task.

Q1. The time axis and a base sinusoid are needed before anything else. [basic]

Your task: Complete `make_axis_and_signal()` so it returns a time vector of N points on [T_MIN, T_MAX] and the signal x(t) = sin(2π·0.5·t) + 0.5·sin(2π·1.5·t).

Template:

```
def make_axis_and_signal():
    t = None    # TODO: N evenly spaced points from T_MIN to T_MAX
    x = None    # TODO: the base signal above
    return t, x
```

Solution. Use `np.linspace` for the axis and vectorized NumPy math for the signal (no loop).

```
def make_axis_and_signal():
    t = np.linspace(T_MIN, T_MAX, N)
    x = np.sin(2*np.pi*0.5*t) + 0.5*np.sin(2*np.pi*1.5*t)
    return t, x
```

`linspace(a, b, N)` includes both endpoints; `arange` would exclude the stop value.

Q2. A discrete index axis is required for discrete-time tasks.

[basic]

Your task: Return the integer index array n = −20, −19, …, 20 (inclusive of both ends).

Template:

```
def make_index():
    n = None    # TODO: integers from -20 to 20 inclusive
    return n
```

Solution. `arange` stops before its second argument, so pass 21 to include 20.

```
def make_index():
    n = np.arange(-20, 21)
    return n
```

Q3. You need the discrete-time unit step u[n] on a given index array. [basic]

Your task: Implement `unit_step(n)` returning 1 where n ≥ 0 and 0 elsewhere, as an integer array.

Template:

```
def unit_step(n):
    # TODO: 1 for n >= 0, else 0
    raise NotImplementedError
```

Solution. A boolean mask cast to int gives exactly this, with no loop.

```
def unit_step(n):
    return (n >= 0).astype(int)
```

`(n >= 0)` is a boolean array; `.astype(int)` turns True/False into 1/0.

Q4. You need the discrete-time unit impulse δ[n]. [basic]

Your task: Implement `unit_impulse(n)` returning 1 only where n == 0.

Template:

```
def unit_impulse(n):
    # TODO: 1 at n == 0, else 0
    raise NotImplementedError
```

Solution. Compare the index array to 0 and cast to int.

```
def unit_impulse(n):
    return (n == 0).astype(int)
```

Q5. A shifted step u[n − n0] is often needed as a building block. [basic]

Your task: Implement `shifted_step(n, n0)` giving u[n − n0].

Template:

```
def shifted_step(n, n0):
    # TODO: unit step delayed by n0
    raise NotImplementedError
```

Solution. Delay means the condition becomes n − n0 ≥ 0.

```
def shifted_step(n, n0):
    return (n - n0 >= 0).astype(int)
```

Positive n0 shifts the step to the right (a delay); negative n0 shifts it left (an advance).

Q6. A real exponential gated by a step is a classic signal. [core]

Your task: Return $x[n] = a^n \cdot u[n]$ for a given base a and index array.

Template:

```
def gated_exponential(n, a):
    # TODO: a**n for n >= 0, else 0
    raise NotImplementedError
```

Solution. Compute the powers, then zero out the negative-index part with the step mask.

```
def gated_exponential(n, a):
    return (a ** n) * (n >= 0)
```

Multiplying by the boolean mask `(n >= 0)` keeps values for $n \geq 0$ and sets the rest to 0.

Part 2 - Time Scaling and Interpolation (Online A style)

Sub-scale a sampled continuous-time signal, filling missing samples by averaging neighbours.

Q7. Time sub-scaling y(t) = x(t/k) needs values of x at points that are not on the sample grid, so you must interpolate. [tricky]

Your task: Implement `interpolate_signal(t_org, x_org, t_query)` so each queried value is the average of the nearest left and right known samples (exact hits return the known value; out-of-range queries return NaN).

Template:

```
def interpolate_signal(t_org, x_org, t_query):
    # TODO: for each query time, average the nearest left & right samples
    raise NotImplementedError
```

Solution. Sort the samples, use `searchsorted` to find the surrounding indices, clip so the edges are safe, average the neighbours, keep exact matches exact, and NaN-out anything outside the range.

```
def interpolate_signal(t_org, x_org, t_query):
    order = np.argsort(t_org)
    ts, xs = t_org[order], x_org[order]
    n = len(ts)
    idx = np.searchsorted(ts, t_query, side="left")
    idc = np.clip(idx, 0, n - 1)
    exact = (idx < n) & np.isclose(ts[idc], t_query)
    left  = xs[np.clip(idx - 1, 0, n - 1)]
    right = xs[np.clip(idx,     0, n - 1)]
    res = np.where(exact, xs[idc], 0.5 * (left + right))
    outside = (t_query < ts[0]) | (t_query > ts[-1])
    return np.where(outside, np.nan, res)
```

The averaging rule is exactly y(1) = 0.5·(x(0) + x(1)) from the task sheet, applied to every query at once.

Q8. With the interpolator ready, produce the sub-scaled signal.

[core]

Your task: Implement `time_scale(t, x, k)` returning y(t) = x(t / k) by querying the existing samples (do not call the analytic formula again).

Template:

```
def time_scale(t, x, k):
    # TODO: y(t) = x(t / k) using interpolate_signal
    raise NotImplementedError
```

Solution. The query points are `t / k`; feed them to the interpolator built from the real samples `(t, x)`.

```
def time_scale(t, x, k):
    return interpolate_signal(t, x, t / k)
```

Re-evaluating `x_of_t(t/k)` would defeat the purpose — the whole point is to interpolate from sampled data.

Q9. You must plot the original and the scaled signal together.

[basic]

Your task: Implement `plot_pair(t, x, y, title)` with both curves, a legend, axis labels and a grid.

Template:

```
def plot_pair(t, x, y, title):
    # TODO: plot x(t) and y(t) on the same axes
    raise NotImplementedError
```

Solution. Standard Matplotlib: two `plot` calls, labels, legend, grid.

```
def plot_pair(t, x, y, title):
    plt.figure(figsize=(10, 5))
    plt.plot(t, x, label="x(t)")
    plt.plot(t, y, label="y(t)", linestyle="--")
    plt.title(title); plt.xlabel("t"); plt.ylabel("Amplitude")
    plt.legend(); plt.grid(True)
```

NaN values from out-of-range queries are skipped automatically by `plot`, leaving a clean gap.

Q10. A colleague wrote the interpolation but it crashes with an IndexError on the largest query values. [tricky]

Your task: Find and fix the bug in the buggy line.

Template:

```
idx = np.searchsorted(ts, t_query)
right = xs[idx]                 # <-- crashes here
left  = xs[idx - 1]
```

Solution. `searchsorted` can return `len(ts)`, which is out of bounds. Clip the indices before using them.

```
idx = np.searchsorted(ts, t_query)
n = len(ts)
right = xs[np.clip(idx,     0, n - 1)]
left  = xs[np.clip(idx - 1, 0, n - 1)]
```

Clipping keeps indexing safe; the truly out-of-range queries are then handled separately (e.g. set to NaN).

Q11. You want linear interpolation (weighted by distance) instead of a plain midpoint average, for smoother results. [tricky]

Your task: Replace the midpoint average with true linear interpolation between the two neighbours. (Assume sorted `ts`, valid interior indices, and left index `i0 = idx - 1`, right index `i1 = idx`.)

Template:

```
# currently: mid = 0.5 * (xs[i0] + xs[i1])
# TODO: weight by how close t_query is to each neighbour
```

Solution. Compute the fractional position of the query between the two sample times and blend accordingly. NumPy even has a built-in for the uniform case.

```
# manual linear interpolation
w = (t_query - ts[i0]) / (ts[i1] - ts[i0])   # 0 at left, 1 at right
val = (1 - w) * xs[i0] + w * xs[i1]

# or, for the whole array at once, the built-in:
val = np.interp(t_query, ts, xs)
```

`np.interp` does sorted-grid linear interpolation directly — handy once you understand what it is doing under the hood.

Part 3 - Time Shift and Phase Change (Online B style)

Discrete sinusoid x[n] = A cos(Omega0 n + phi); relate shifting in time to changing phase.

Q12. The base sinusoid must be implemented first. [basic]

Your task: Implement `sinusoid(n, A, Omega0, phi)` = $A\cos(\Omega_0 n + \varphi)$.

Template:

```
def sinusoid(n, A, Omega0, phi):
    raise NotImplementedError
```

Solution. A single vectorized cosine expression.

```
def sinusoid(n, A, Omega0, phi):
    return A * np.cos(n * Omega0 + phi)
```

Q13. A time shift moves the signal along the index axis. [basic]

Your task: Implement `time_shift_sinusoid(n, A, Omega0, phi, n0)` giving x[n + n0].

Template:

```
def time_shift_sinusoid(n, A, Omega0, phi, n0):
    raise NotImplementedError
```

Solution. Reuse `sinusoid` with the index replaced by n + n0.

```
def time_shift_sinusoid(n, A, Omega0, phi, n0):
    return sinusoid(n + n0, A, Omega0, phi)
```

If your course defines the shift as a delay x[n − n0], use `n - n0` instead — keep it consistent with the phase formula below.

Q14. A phase change adds to the angle rather than the index.

[basic]

Your task: Implement `phase_change_sinusoid(n, A, Omega0, phi, phi0)` that adds φ_0.

Template:

```
def phase_change_sinusoid(n, A, Omega0, phi, phi0):
    raise NotImplementedError
```

Solution. Add phi0 to the existing phase argument.

```
def phase_change_sinusoid(n, A, Omega0, phi, phi0):
    return sinusoid(n, A, Omega0, phi + phi0)
```

Q15. Part A of the online asks for the phase change that exactly matches a given time shift. [core]

Your task: Fill in `phi0_equiv` so that a shift of n0 samples is reproduced as a phase change.

Template:

```
n0 = 3
x_time = time_shift_sinusoid(n, A, Omega0, phi, n0)
phi0_equiv = None          # TODO
x_phase = phase_change_sinusoid(n, A, Omega0, phi, phi0_equiv)
```

Solution. Expanding the cosine shows the equivalent phase is $\Omega_0 \cdot n0$.

```
phi0_equiv = Omega0 * n0
```

Because $\cos(\Omega_0(n+n0)+\varphi) = \cos(\Omega_0 n + \varphi + \Omega_0 n0)$, the two signals are identical and their MSE is ~0.

Q16. Part B searches integer shifts for the one that best matches an arbitrary phase change. [core]

Your task: Complete the search loop so it records the integer k in [k_min, k_max] with the smallest MSE against x_phase.

Template:

```
best_k, best_err = None, None
for k in range(k_min, k_max + 1):
    x_time_k = time_shift_sinusoid(n, A, Omega0, phi, k)
    e = mse(x_time_k, x_phase)
    # TODO: keep the best k
```

Solution. Update the best pair whenever a strictly smaller error appears.

```
    if best_err is None or e < best_err:
        best_err = e
        best_k = k
```

The strict < means that among ties (shifts differing by a full period), the first one encountered is kept.

Q17. You must implement the mean-squared-error helper used by the search. [basic]

Your task: Implement `mse(a, b)` for two equal-length arrays.

Template:

```
def mse(a, b):
    # TODO: mean of squared differences, as a float
    raise NotImplementedError
```

Solution. Square the element-wise difference and take the mean.

```
def mse(a, b):
    return float(np.mean((a - b) ** 2))
```

Q18. For $\Omega_0 = \pi/4$, the search unexpectedly returns k = −8 rather than k = 0, and you must explain it in a comment. [tricky]

Your task: Write the one-line reason (as a comment) for why k = 0 and k = ±8 tie.

Template:

```
# best_k came back as -8, not 0. Why the tie?
# TODO: explain in one line
```

Solution. The period in samples is $2\pi/\Omega_0 = 8$, so shifting by any multiple of 8 leaves the cosine unchanged.

```
# 8 * Omega0 = 8 * (pi/4) = 2*pi, a full period, so shifts of
# 0, +-8, +-16, ... give the identical signal and equal (minimum) MSE.
```

Q19. You want to overlay the two Part-A signals as stem plots to show they coincide. [core]

Your task: Implement `stem_pair(n, x1, x2, l1, l2)` that stem-plots both on one axis with a legend and grid, and hides the ugly baseline.

Template:

```
def stem_pair(n, x1, x2, l1, l2):
    # TODO: two stem plots on one axis
    raise NotImplementedError
```

Solution. `stem` returns three handles; grab the baseline to hide it.

```
def stem_pair(n, x1, x2, l1, l2):
    fig, ax = plt.subplots(figsize=(9, 4))
    for x, lab in [(x1, l1), (x2, l2)]:
        _, _, base = ax.stem(n, x, label=lab)
        base.set_visible(False)
    ax.set_xlabel("n"); ax.set_ylabel("Amplitude")
    ax.legend(); ax.grid(True, alpha=0.3)
```

Part 4 - Time Reversal and Even / Odd Decomposition (Online C style)

Reverse a sampled signal, then split it into even and odd parts using only your reversal function.

Q20. Time reversal x(−t) is the first building block. [core]

Your task: Implement `time_reverse(x)` that returns the samples of x(−t) on a symmetric grid, without mutating the input.

Template:

```
def time_reverse(x):
    # TODO: return samples in reversed order; do NOT change x
    raise NotImplementedError
```

Solution. On a symmetric grid, reversal is flipping the array. Copy so the original is untouched.

```
def time_reverse(x):
    return x[::-1].copy()
```

The swap-in-place version works too, but you must start from `xr = x.copy()` or you corrupt the caller's array.

Q21. Here is a swap-based reversal that produces wrong even/odd results. It reverses correctly the first time but the original signal gets destroyed. [tricky]

Your task: Fix the single line responsible for the aliasing bug.

Template:

```
def time_reverse(x):
    mid = (len(x) - 1) / 2.0
    xr = x                        # <-- bug
    i = 0
    while i <= mid:
        xr[i], xr[len(x)-1-i] = x[len(x)-1-i], x[i]
        i += 1
    return xr
```

Solution. `xr = x` makes xr an alias, so the in-place swaps also edit x. Copy first.

```
    xr = x.copy()               # independent array
```

With the alias, even_odd_decompose computes x − xr on two names for the same data → the odd part becomes all zeros.

Q22. Now decompose any signal into even and odd parts. [core]

Your task: Implement `even_odd_decompose(x)` using only `time_reverse`, returning (xe, xo).

Template:

```
def even_odd_decompose(x):
    # TODO: must call time_reverse(...) inside
    raise NotImplementedError
```

Solution. Even part is the half-sum with its reversal; odd part is the half-difference.

```
def even_odd_decompose(x):
    xr = time_reverse(x)
    xe = 0.5 * (x + xr)
    xo = 0.5 * (x - xr)
    return xe, xo
```

Q23. The instructor wants numeric proof that your decomposition is correct. [core]

Your task: Write three checks that print numbers close to 0: xe is even, xo is odd, and xe + xo reconstructs x.

Template:

```
xe, xo = even_odd_decompose(x)
# TODO: three verification prints
```

Solution. Use the defining symmetries and the reconstruction identity.

```
print(np.max(np.abs(xe - time_reverse(xe))))   # even  -> ~0
print(np.max(np.abs(xo + time_reverse(xo))))   # odd   -> ~0
print(np.max(np.abs(x - (xe + xo))))           # recon -> ~0
```

Q24. You must plot x(t), xe(t) and xo(t) on one figure. [basic]

Your task: Implement `plot_three(t, x, xe, xo)` with three labelled curves, title, axis labels, legend and grid.

Template:

```
def plot_three(t, x, xe, xo):
    # TODO
    raise NotImplementedError
```

Solution. Three plot calls with labels, then the usual decorations.

```
def plot_three(t, x, xe, xo):
    plt.figure()
    plt.plot(t, x,  label="x(t)")
    plt.plot(t, xe, label="xe(t)")
    plt.plot(t, xo, label="xo(t)")
    plt.title("Even-Odd Decomposition")
    plt.xlabel("t"); plt.ylabel("Amplitude")
    plt.grid(True); plt.legend()
```

Q25. The program runs but no window ever appears. [basic]

Your task: State the missing final call and where it goes.

Template:

```
def main():
    t = np.linspace(T_MIN, T_MAX, N)
    x = x_of_t(t)
    xr = time_reverse(x)
    xe, xo = even_odd_decompose(x)
    plot_three(t, x, xe, xo)
    plot_pair(t, x, xr)
    # ??? nothing shows
```

Solution. The figures are built but never displayed; add `plt.show()` at the end of `main`.

```
    plt.show()
```

Part 5 - Step / Impulse Synthesis and Relationships

Recreate the identities from the lecture in code.

Q26. The impulse is the first difference of the step. [core]

Your task: Given u = unit_step(n), compute δ[n] = u[n] − u[n − 1] using `np.diff` style logic (keep the same length).

Template:

```
u = unit_step(n)
delta = None    # TODO: first difference, same length as u
```

Solution. Prepend the first element before differencing so the length is preserved.

```
delta = np.diff(u, prepend=u[0])
```

`prepend=u[0]` makes the first difference 0 at the left edge, matching u[n] − u[n−1] there.

Q27. The step is the running sum of the impulse. [core]

Your task: Given `delta` = unit_impulse(n), reconstruct the step by a cumulative sum.

Template:

```
delta = unit_impulse(n)
u = None    # TODO: running sum reproduces the step
```

Solution. `np.cumsum` accumulates left to right, turning the single spike into a step.

```
u = np.cumsum(delta)
```

Q28. Build the step from delayed impulses. [core]

Your task: For index array n and a horizon K, construct u[n] = $\sum_{k=0}^{K} \delta[n-k]$ and compare to the direct step.

Template:

```
K = 30
acc = np.zeros_like(n)
# TODO: add delayed impulses delta[n-k] for k = 0..K
```

Solution. Loop k from 0 to K, adding each shifted impulse; the sum fills all positions n ≥ 0.

```
for k in range(K + 1):
    acc = acc + unit_impulse(n - k)
# acc now equals unit_step(n) for n <= K
```

Vectorized alternative: `acc = ((n >= 0) & (n <= K)).astype(int)`.

Q29. You need a rectangular pulse that is 1 on an interval and 0 outside. [basic]

Your task: Return a pulse that equals 1 for a ≤ n ≤ b (inclusive) and 0 elsewhere.

Template:

```
def rect_pulse(n, a, b):
    # TODO: 1 on [a, b], else 0
    raise NotImplementedError
```

Solution. Combine two conditions with & (element-wise AND), parenthesized.

```
def rect_pulse(n, a, b):
    return ((n >= a) & (n <= b)).astype(int)
```

Using and here would raise 'truth value of an array is ambiguous'; arrays need &.

Q30. Represent a weighted sum of shifted impulses: x[n] = 2δ[n−1] − 3δ[n−2] + 2δ[n−4]. [core]

Your task: Build this signal on the index array n.

Template:

```
x = np.zeros_like(n, dtype=float)
# TODO: place the three weighted impulses
```

Solution. Add each weighted, shifted impulse.

```
x = (2*unit_impulse(n-1)
     - 3*unit_impulse(n-2)
     + 2*unit_impulse(n-4)).astype(float)
```

Part 6 - Shift, Reversal and Scaling of the Index

Combined transformations x[an + b] and their correct order of operations.

Q31. You must implement a pure time shift on sampled data (discrete). [tricky]

Your task: Return y[n] = x[n − n0] for integer n0, padding vacated positions with 0 (same length as x).

Template:

```
def shift(x, n0):
    # TODO: delay by n0, zero-fill, keep length
    raise NotImplementedError
```

Solution. `np.roll` rotates; instead build a zero array and copy the overlapping slice so shifted-in samples are 0.

```
def shift(x, n0):
    y = np.zeros_like(x)
    if n0 >= 0:
        y[n0:] = x[:len(x)-n0] if n0 < len(x) else y[n0:]
    else:
        y[:n0] = x[-n0:]
    return y
```

`np.roll` would wrap end-around, which is usually not what a time shift means for a finite signal.

Q32. Time reversal of a sequence that is not on a symmetric grid. [basic]

Your task: Reverse the sample order of x (this represents x[−n] when indices are stored separately).

Template:

```
def reverse_seq(x):
    # TODO
    raise NotImplementedError
```

Solution. Slice with a step of −1 and copy.

```
def reverse_seq(x):
    return x[::-1].copy()
```

Q33. A combined transformation y[n] = x[−n + 2] is requested, and order of operations matters. [tricky]

Your task: Produce y from x on index array n by first reversing then shifting (state the order in a comment).

Template:

```
# want y[n] = x[-n + 2] = x[-(n - 2)]
# TODO: reverse, then shift by +2
```

Solution. Rewrite as x[−(n − 2)]: shift the argument by 2, then reverse. In sample terms, reverse the array and then delay.

```
xr = reverse_seq(x)      # x[-n]
y  = shift(xr, 2)        # x[-(n-2)] = x[-n+2]
```

Always factor the argument as x[−(n − n0)] to read off the reversal and the shift unambiguously.

Q34. Downsampling keeps every k-th sample. [basic]

Your task: Implement `downsample(x, k)` returning x[0], x[k], x[2k], ...

Template:

```
def downsample(x, k):
    # TODO: every k-th sample
    raise NotImplementedError
```

Solution. A slice with step k.

```
def downsample(x, k):
    return x[::k]
```

Q35. Upsampling by k inserts k−1 zeros between samples. [core]

Your task: Implement `upsample(x, k)` so the output length is k·len(x) with zeros in the gaps.

Template:

```
def upsample(x, k):
    # TODO: place x values at 0, k, 2k, ... else 0
    raise NotImplementedError
```

Solution. Make a zero array of the new length and assign the originals into every k-th slot.

```
def upsample(x, k):
    y = np.zeros(len(x) * k, dtype=x.dtype)
    y[::k] = x
    return y
```

Part 7 - Energy, Power, Convolution and Correlation

Standard numeric summaries and the operation at the heart of LTI systems.

Q36. Signal energy is the sum of squared magnitudes. [basic]

Your task: Implement `energy(x)` for a real discrete signal.

Template:

```
def energy(x):
    # TODO: sum of squares
    raise NotImplementedError
```

Solution. Square and sum, returned as a float.

```
def energy(x):
    return float(np.sum(x ** 2))
```

Q37. Average power over the given samples. [basic]

Your task: Implement `average_power(x)` = mean of the squared values.

Template:

```
def average_power(x):
    # TODO
    raise NotImplementedError
```

Solution. The mean of the squares.

```
def average_power(x):
    return float(np.mean(x ** 2))
```

Q38. Classify a signal as (finite-)energy or power. [tricky]

Your task: Return the string "energy" if total energy is finite and non-trivial while average power → 0 over the window, else "power". (Use a simple threshold on the given finite window.)

Template:

```
def classify(x):
    E = energy(x)
    P = average_power(x)
    # TODO: return "energy" or "power" based on E and P
    raise NotImplementedError
```

Solution. On a finite window this is heuristic: a decaying/finite-support signal has modest E and tiny P; a persistent one has large E that grows with N but finite P.

```
def classify(x):
    E = energy(x); P = average_power(x)
    return "energy" if P < 1e-6 and E < np.inf else "power"
```

Formally the distinction is a limit as N → ∞; on a fixed window we can only approximate it, so treat this as illustrative.

Q39. Discrete convolution combines a signal with an impulse response. [core]

Your task: Implement `convolve(x, h)` for the full linear convolution.

Template:

```
def convolve(x, h):
    # TODO: full linear convolution
    raise NotImplementedError
```

Solution. NumPy provides it directly; the output length is len(x) + len(h) − 1.

```
def convolve(x, h):
    return np.convolve(x, h, mode="full")
```

`mode="same"` returns a result the length of x; `"valid"` only the fully-overlapping part.

Q40. Convolving any signal with a unit impulse must return the signal unchanged. [core]

Your task: Verify `x * δ = x` numerically for a test signal.

Template:

```
x = np.array([2.0, -1.0, 3.0, 0.5])
d = np.array([1.0])              # unit impulse
# TODO: convolve and compare to x
```

Solution. Convolving with the length-1 impulse leaves x unchanged.

```
y = np.convolve(x, d, mode="full")
print(np.allclose(y, x))        # True
```

Convolving with a shifted impulse `[0, 1]` would delay x by one sample — the sifting property in action.

Q41. A moving-average smoother is a convolution with a boxcar. [core]

Your task: Build a length-L normalized averaging filter and smooth x with it (same length output).

Template:

```
def moving_average(x, L):
    # TODO: convolve with a length-L box of height 1/L
    raise NotImplementedError
```

Solution. Make the kernel `np.ones(L)/L` and convolve in 'same' mode.

```
def moving_average(x, L):
    h = np.ones(L) / L
    return np.convolve(x, h, mode="same")
```

Q42. Cross-correlation measures similarity at various lags. [core]

Your task: Compute the cross-correlation of `x` and `y` with `np.correlate` (full).

Template:

```
def cross_corr(x, y):
    # TODO: full cross-correlation
    raise NotImplementedError
```

Solution. Use `np.correlate` in full mode.

```
def cross_corr(x, y):
    return np.correlate(x, y, mode="full")
```

Auto-correlation is just `cross_corr(x, x)`; its peak sits at zero lag.

Part 8 - Numerically Testing System Properties

Write small experiments that check linearity, time-invariance and causality of a given system.

Q43. You are given a system as a Python function `S(x)` mapping an input array to an output array, and must test linearity numerically. [core]

Your task: Implement `is_linear(S, x1, x2, a, b)` that checks S(a·x1 + b·x2) ≈ a·S(x1) + b·S(x2).

Template:

```
def is_linear(S, x1, x2, a, b):
    # TODO: compare superposition of outputs
    raise NotImplementedError
```

Solution. Compute both sides and compare with `np.allclose`.

```
def is_linear(S, x1, x2, a, b):
    lhs = S(a * x1 + b * x2)
    rhs = a * S(x1) + b * S(x2)
    return bool(np.allclose(lhs, rhs))
```

A single random pair (x1, x2) with random a, b is usually enough to disprove linearity; passing does not prove it, but is strong evidence.

Q44. Test time-invariance of a system numerically. [tricky]

Your task: Implement `is_time_invariant(S, x, n0)`: shifting the input by n0 should shift the output by n0.

Template:

```
def is_time_invariant(S, x, n0):
    # TODO: compare S(shift(x)) with shift(S(x))
    raise NotImplementedError
```

Solution. Apply the system to the shifted input, and separately shift the system's output; compare on the region that is unaffected by zero-fill edges.

```
def is_time_invariant(S, x, n0):
    lhs = S(shift(x, n0))
    rhs = shift(S(x), n0)
    # ignore the first n0 edge samples affected by zero-fill
    return bool(np.allclose(lhs[n0:], rhs[n0:]))
```

Q45. Test causality by checking that the output at index i does not depend on future inputs. [tricky]

Your task: Implement `depends_on_future(S, x, i)`: perturb only x[i+1:] and see whether S(x)[i] changes.

Template:

```
def depends_on_future(S, x, i):
    # TODO: change a future sample, check output at i
    raise NotImplementedError
```

Solution. Copy x, perturb a strictly-future sample, and compare the i-th outputs.

```
def depends_on_future(S, x, i):
    x2 = x.copy()
    if i + 1 < len(x):
        x2[i + 1] += 1.0
    return not np.isclose(S(x)[i], S(x2)[i])
```

If this returns True for some i, the system used a future input at that index → it is non-causal.

Part 9 - Multi-panel Plotting Tasks

Assemble a 2x2 figure with the four standard chart types, correctly labelled.

Q46. A line-plot panel must show three series with distinct line styles. [core]

Your task: Fill in three `plot` calls: solid blue, dashed green, dotted red, each labelled.

Template:

```
ax = grd[0, 0]
# TODO: three styled line plots (north/south/central)
ax.set_title("Monthly Profit"); ax.set_xlabel("Month")
ax.set_ylabel("Profit"); ax.legend(); ax.grid(True)
```

Solution. Use `linestyle` (not `marker`) for the line patterns.

```
ax.plot(months, north,   linestyle="-",  color="blue",  label="North")
ax.plot(months, south,   linestyle="--", color="green", label="South")
ax.plot(months, central, linestyle=":",  color="red",   label="Central")
```

`"-"`, `"--"`, `":"` are line styles; passing them to `marker=` is a common mistake.

Q47. A bar panel needs one colour per bar — and a colleague's code silently used wrong bar widths. [core]

Your task: Fix the bar call so the colour list is applied as colours, not widths.

Template:

```
colors_ = ["blue", "green", "red"]
grd[0, 1].bar(branches, dec_profit, colors_)   # <-- wrong
```

Solution. The third positional argument of `bar` is width; pass colour by keyword.

```
grd[0, 1].bar(branches, dec_profit, color=colors_)
```

Q48. A scatter panel must use a specific marker size, and the given code crashes. [basic]

Your task: Fix the invalid keyword.

Template:

```
grd[1, 0].scatter(months, north, color="blue", size=60)  # crashes
```

Solution. The size keyword for `scatter` is `s`, not `size`.

```
grd[1, 0].scatter(months, north, color="blue", s=60)
```

Q49. A stacked-bar panel of quarterly profits stacks incorrectly because list addition concatenates. [tricky]

Your task: Fix the quarterly arrays and the `bottom` so segments stack properly.

Template:

```
north_q = [sum(north[:3]), sum(north[3:6]), sum(north[6:9]), sum(north[9:])]
south_q = [sum(south[:3]), sum(south[3:6]), sum(south[6:9]), sum(south[9:])]
grd[1,1].bar(q, north_q, color="blue", label="North")
grd[1,1].bar(q, south_q, bottom=north_q, color="green", label="South")
grd[1,1].bar(q, central_q, bottom=north_q + south_q, ...)   # concatenates!
```

Solution. Make the quarterly values NumPy arrays so `+` adds element-wise; `reshape` is the cleanest way to build them.

```
north_q   = north.reshape(4, 3).sum(axis=1)
south_q   = south.reshape(4, 3).sum(axis=1)
central_q = central.reshape(4, 3).sum(axis=1)
grd[1,1].bar(q, north_q,   color="blue",  label="North")
grd[1,1].bar(q, south_q,   bottom=north_q,           color="green", label="South")
grd[1,1].bar(q, central_q, bottom=north_q + south_q, color="red",   label="Central")
```

With Python lists, `north_q + south_q` makes an 8-element concatenation; arrays give the 4-element element-wise sum you need.

Q50. The whole figure needs one overall title without overlapping the panels. [core]

Your task: Add the main title and lay out the panels so nothing overlaps.

Template:

```
fig, grd = plt.subplots(2, 2, figsize=(14, 10))
# ... four panels filled in ...
# TODO: main title + neat layout + show
```

Solution. `suptitle` for the big title, then reserve top space in `tight_layout`.

```
fig.suptitle("Company Branch Performance Analysis (2025)", fontsize=16)
plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

Without the `rect`, a plain `tight_layout()` can let the suptitle overlap the top row of titles.

Part 10 - Mixed Exam-style Tasks

Slightly larger prompts that combine several of the skills above.

Q51. Full Online-A-style task: given `x_of_t`, produce and plot y(t) = x(t/3). [core]

Your task: Fill both stubs so `main` runs end to end (reuse `interpolate_signal` from Part 2).

Template:

```
def time_scale(t, x, k):
    return None          # TODO

def main():
    t = np.linspace(T_MIN, T_MAX, N)
    x = x_of_t(t)
    y = time_scale(t, x, 3)
    plot_pair(t, x, y, "y(t) = x(t/3)")
    # TODO: display
```

Solution. Query the samples at t/k and remember `plt.show()`.

```
def time_scale(t, x, k):
    return interpolate_signal(t, x, t / k)

# ... at the end of main():
    plt.show()
```

Q52. Full Online-B-style task: print both MSEs and pick the best integer shift for a phase change of π/15. [core]

Your task: Complete the two TODOs (equivalent phase, and best-shift bookkeeping).

Template:

```
phi0_equiv = None                          # TODO (Part A)
x_phase_equiv = phase_change_sinusoid(n, A, Omega0, phi, phi0_equiv)
print("[A] MSE:", mse(x_time, x_phase_equiv))

best_k = best_err = None
for k in range(-12, 13):
    e = mse(time_shift_sinusoid(n, A, Omega0, phi, k), x_phase)
    # TODO (Part B): track best
```

Solution. Equivalent phase is $\Omega_0 \cdot n0$; track the strictly-smallest error.

```
phi0_equiv = Omega0 * n0
# inside the loop:
    if best_err is None or e < best_err:
        best_err, best_k = e, k
```

Q53. Full Online-C-style task: reverse the signal and plot x, xe, xo plus a separate x vs x(−t). [basic]

Your task: Fill the three TODOs in `main`.

Template:

```
def main():
    t = np.linspace(T_MIN, T_MAX, N)
    x  = None                  # TODO
    xr = None                  # TODO
    xe, xo = None              # TODO
    plot_three(t, x, xe, xo)
    plot_pair(t, x, xr)
    plt.show()
```

Solution. Build x from `x_of_t`, reverse it, and decompose.

```
    x  = x_of_t(t)
    xr = time_reverse(x)
    xe, xo = even_odd_decompose(x)
```

Q54. Given a periodic-looking discrete sinusoid, decide programmatically whether it is periodic and find the period.

[tricky]

Your task: Implement `discrete_period(Omega0, max_N=1000)` returning the smallest positive integer period N, or None if none is found up to max_N.

Template:

```
def discrete_period(Omega0, max_N=1000):
    # periodic iff Omega0*N is a multiple of 2*pi for some integer N
    # TODO
    raise NotImplementedError
```

Solution. Search N = 1..max_N for one where $\Omega_0 N/(2\pi)$ is (numerically) an integer.

```
def discrete_period(Omega0, max_N=1000):
    for N in range(1, max_N + 1):
        ratio = Omega0 * N / (2 * np.pi)
        if np.isclose(ratio, round(ratio)):
            return N
    return None
```

For $\Omega_0 = 16\pi/63$ this returns 63; for $\Omega_0 = 2$ it returns None (cos(2n) is not periodic).

Q55. Combine convolution and plotting: filter a noisy signal and show before/after. [core]

Your task: Fill the TODOs to smooth x with a length-5 moving average and plot both.

Template:

```
L = 5
y = None                        # TODO: smoothed signal
plt.plot(n, x, label="noisy")
# TODO: plot smoothed, add labels/legend/grid, show
```

Solution. Reuse the moving-average filter and add the usual plot decorations.

```
y = moving_average(x, L)
plt.plot(n, x, label="noisy")
plt.plot(n, y, label="smoothed")
plt.xlabel("n"); plt.ylabel("Amplitude")
plt.legend(); plt.grid(True); plt.show()
```

Q56. Reconstruct a signal from its even and odd parts to confirm the decomposition round-trips. [basic]

Your task: Given xe, xo, rebuild x and report the maximum absolute error against the original.

Template:

```
# xe, xo already computed from x
x_rebuilt = None                # TODO
err = None                      # TODO: max abs difference from x
```

Solution. Sum the parts and take the max absolute deviation.

```
x_rebuilt = xe + xo
err = float(np.max(np.abs(x - x_rebuilt)))
print(err)                        # ~0
```

Q57. Final integrative task: given monthly data for three branches, compute quarterly totals and identify the best branch per quarter. [tricky]

Your task: Fill the TODOs: quarterly sums (4x3 reshape) and the per-quarter argmax over the three branches.

Template:

```
data = np.vstack([north, south, central])   # shape (3, 12)
names = np.array(["North", "South", "Central"])
quarterly = None            # TODO: shape (3, 4) quarterly sums per branch
best_idx  = None            # TODO: index of top branch each quarter
best      = names[best_idx]
```

Solution. Reshape each row to (4,3) and sum; stack to (3,4); then argmax down the branch axis.

```
quarterly = data.reshape(3, 4, 3).sum(axis=2)   # (3 branches, 4 quarters)
best_idx  = quarterly.argmax(axis=0)           # top branch per quarter
best      = names[best_idx]
print(best)
```

`reshape(3, 4, 3)` groups each branch's 12 months into 4 quarters of 3; `argmax(axis=0)` compares across branches for each quarter.